

Summary

- ① CM5
- ② Overview
- ③ Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- ④ Short Summary on Event-B
- ⑤ Internalizing events by merging
- ⑥ The Call-as-Event Paradigm
- ⑦ Analysis and then Synthesis
 - Analysis using Refinement
 - Synthesis by Merging
- ⑧ Summing a list of integer values

Current Section Summary

Current Section Summary

- 1 CM5
- 2 Overview
- 3 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
 - Analysis using Refinement
 - Synthesis by Merging
- 8 Summing a list of integer values

Introduction

- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.

- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.
- Possibility to combine with others provers as TLAPS, Frama-c

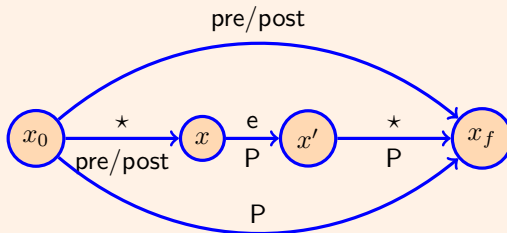
- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.
- Possibility to combine with others provers as TLAPS, Frama-c
- Illustrating the merging of events and the step to code.

- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.
- Possibility to combine with others provers as TLAPS, Frama-c
- Illustrating the merging of events and the step to code.
- Relationship to PlusCal and TLA⁺ proof tools

Big steps versus Small steps

From pre/post to algorithm(s)

Refining a pre/post specification into a sequential algorithm



Current Section Summary

- ① CM5
- ② Overview
- ③ Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- ④ Short Summary on Event-B
- ⑤ Internalizing events by merging
- ⑥ The Call-as-Event Paradigm
- ⑦ Analysis and then Synthesis
 - Analysis using Refinement
 - Synthesis by Merging
- ⑧ Summing a list of integer values

- 1 CM5
- 2 Overview
- 3 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
- 8 Summing a list of integer values

Verification of contract (I)

A program P *satisfies* a contract $(\text{pre}, \text{post})$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Floyd style

```
requires  $\text{pre}(x_0)$ 
ensures  $\text{post}(x_0, x_f)$ 
variables  $X$ 
begin
  0 :  $P_0(x_0, x)$ 
  instruction0
  ...
  i :  $P_i(x_0, x)$ 
  ...
  instructionf-1
  f :  $P_f(x_0, x)$ 
end
```

- $\text{pre}(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$
- $\text{pre}(x_0) \wedge P_f(x_0, x) \Rightarrow \text{post}(x_0, x)$
- For each pair ℓ, ℓ'
such that $\ell \longrightarrow \ell'$, one checks that
for any value $x, x' \in \text{MEMORY}$
$$\left(\begin{array}{l} \text{pre}(x_0) \wedge P_\ell(x_0, x) \\ \wedge \text{cond}_{\ell, \ell'}(x) \wedge x' = f_{\ell, \ell'}(x) \end{array} \right) \Rightarrow P_{\ell'}(x_0, x')$$

Verification du contract (II)

A program P *satisfies* a contract $(pre, post)$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $pre(x_0)$ and x_f satisfait $post$: $post(x_0, x_f)$
- $pre(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow post(x_0, x_f)$

Hoare style

```
requires  $pre(x_0)$   
ensures  $post(x_0, x_f)$   
variables  $X$   
[  
  begin  
  0 :  $P_0(x_0, x)$   
  instruction0  
  ...  
  i :  $P_i(x_0, x)$   
  ...  
  instructionf-1  
  f :  $P_f(x_0, x)$   
end
```

Verification du contract (II)

A program P *satisfies* a contract (pre,post) :

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Hoare style

requires $\text{pre}(x_0)$
ensures $\text{post}(x_0, x_f)$
variables X

```
[ begin  
  0 :  $P_0(x_0, x)$   
  instruction0  
  ...  
  i :  $P_i(x_0, x)$   
  ...  
  instructionf-1  
  f :  $P_f(x_0, x)$   
end
```

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Verification du contract (II)

A program P *satisfies* a contract (pre,post) :

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Hoare style

```
requires pre(x0)  
ensures post(x0, xf)  
variables X
```

```
[  
  begin  
  0 : P0(x0, x)  
  instruction0  
  ...  
  i : Pi(x0, x)  
  ...  
  instructionf-1  
  f : Pf(x0, x)  
end
```

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$

Verification du contract (II)

A program P *satisfies* a contract (pre,post) :

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Hoare style

requires $\text{pre}(x_0)$
ensures $\text{post}(x_0, x_f)$
variables X

```
begin  
  0 :  $P_0(x_0, x)$   
  instruction0  
  ...  
  i :  $P_i(x_0, x)$   
  ...  
  instructionf-1  
  f :  $P_f(x_0, x)$   
end
```

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x_f. (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$

Verification du contract (II)

A program P *satisfies* a contract (pre,post) :

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Hoare style

```
requires pre(x0)
ensures post(x0, xf)
variables X
[
  begin
  0 : P0(x0, x)
  instruction0
  ...
  i : Pi(x0, x)
  ...
  instructionf-1
  f : Pf(x0, x)
  end
```

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x_f. (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x. (x_0 \xrightarrow{P} x \Rightarrow \text{post}(x_0, x))$

Verification du contract (II)

A program P *satisfies* a contract $(\text{pre}, \text{post})$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Hoare style

```
requires  $\text{pre}(x_0)$ 
ensures  $\text{post}(x_0, x_f)$ 
variables  $X$ 
[
  begin
  0 :  $P_0(x_0, x)$ 
  instruction0
  ...
  i :  $P_i(x_0, x)$ 
  ...
  instructionf-1
  f :  $P_f(x_0, x)$ 
end
```

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x_f. (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x. (x_0 \xrightarrow{P} x \Rightarrow \text{post}(x_0, x))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow WLP(P)(\text{post}(x_0, x))$

Listing 1 – Simple contract

```
#ifndef _A_H
#define _A_H
// Definition of the mathematical function mathpower2
/*@ axiomatic auxmath {
  @ lemma rule1: \forall int n; n > 0 ==> n*n == (n-1)*(n-1)+2*(n-1)+1;
  @ } */
/*@ requires 0 <= x;
  @ requires x*x <= INT_MAX;
  assigns \nothing;
  ensures \result == x*x;
*/
int power2(int x);
#endif
```

Listing 2 – Simple contract

```
#include <limits.h>
#include "power2.h"

int power2(int x)
{
    int r, k, cv, cw, or, ok, ocv, ocw;
    r=0; k=0; cv=0; cw=0; or=0; ok=k; ocv=cv; ocw=cw;
    /*@ loop invariant 0 <= cv && 0 <= cw && 0 <= k;
       @ loop invariant cv == k*k;
       @ loop invariant k <= x;
       @ loop invariant cw == 2*k;
       @ loop invariant 4*cv == cw*cw;
       @ loop assigns k, cv, cw, or, ok, ocv, ocw;
       @ loop variant x-k;
    */
    while (k<x)
    {
        ok=k; ocv=cv; ocw=cw;
        k=ok+1;
        cv=ocv+ocw+1;
        cw=ocw+2;
    }
    r=cv;
    return(r);
}
```

A Simple C Function

Listing 3 – Simple contract

```
frama-c -wp -rte power2.c
[kernel] Parsing power2.c (with preprocessing)
[rte:annot] annotating function power2
[wp] 30 goals scheduled
[wp] Proved goals: 32 / 32
Terminating: 1
Unreachable: 1
Qed: 22 (5ms-1ms-7ms)
Alt-Ergo 2.6.0: 8 (13ms-15ms-18ms)
```

- Finding or computing annotations is difficult and even undecidable!

- Finding or computing annotations is difficult and even undecidable!

- Given

Contract EX
 Variables $X(int)$
 Requires $pre(x_0)$
 Ensures $post(x_0, x_f)$

Correct by Construction

- Finding or computing annotations is difficult and even undecidable!

- Given

Contract EX
 Variables $X(int)$
 Requires $pre(x_0)$
 Ensures $post(x_0, x_f)$

- Design of the algorithm ALG fulfilling the contract

- Finding or computing annotations is difficult and even undecidable !

- Given

Contract EX Variables $X(int)$ Requires $pre(x_0)$ Ensures $post(x_0, x_f)$
--

- Design of the algorithm ALG fulfilling the contract
- HOARE triple :

$$\{\mathbf{pre}(\mathbf{x}_0) \wedge \mathbf{x} = \mathbf{x}_0\} \text{ALG} \{\mathbf{post}(\mathbf{x}_0, \mathbf{x})\}$$

- Given

Contract EX
 Variables $X(int)$
 Requires $pre(x_0)$
 Ensures $post(x_0, x_f)$

- HOARE triple :

$$\{\text{pre}(\mathbf{x}_0) \wedge \mathbf{x} = \mathbf{x}_0\} \text{ALG} \{\text{post}(\mathbf{x}_0, \mathbf{x})\}$$

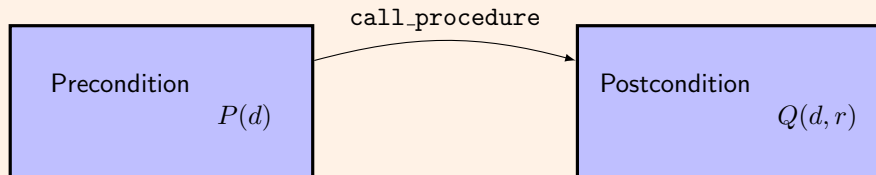
Idea ?

From Contract *EX*, using a step by step approach to find an algorithm
ALG satisfying it using refinement and Event-B models.
Applying the CLEANROOM model but with tool support

- 1 CM5
- 2 Overview
- 3 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
- 8 Summing a list of integer values

- Providing patterns for helping the use of refinement.
- Making proofs as simple as possible.
- Simplifying the discovery of invariants
- we focus on the transformation of the final concrete specification into a PlusCal algorithm :
 - ① transforming an `EVENT B` specification into a PlusCal algorithm
 - ② using the merging of events to get only one event corresponding to the call of the algorithm
 - ③ double checking the resulting algorithm by using TLA^+ and its proof system.

The call as event idea



- `call_procedure` is simulated by the event `call_procedure` :

```
EVENT CALL_PROCEDURE
WHEN
   $P(d)$ 
THEN
   $r : |Q(d, r')$ 
END
```

- $P(d)$ is specifying the inputs of the procedure.
- $Q(d, r)$ is expressing the relation between the inputs d and the results r .

- 1 CM5
- 2 Overview
- 3 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
- 8 Summing a list of integer values

- A pre/post specification is a pair of assertions $(pre, post)$
- $(pre, post)$ is defining a *transformation* over variables that is correct with respect to *HOARE* triples :

$$\{pre\} transformation \{post\}$$

- **Problem** : Design of a program PROG simulating the *transformation* with respect to $(pre, post)$:

$$\{pre\} PROG \{post\}$$

- **Idea** : Writing a *one-shot* relation simulating the relation over *flexible* variables :

$$\forall x, y. pre(x) \wedge R(x, y) \Rightarrow post(x, y)$$

- **event** : Expressing the relation :

```
EVENT  PROG
WHEN
  pre(x)
THEN
  y : |post(x, y')
END
```

The sequential pattern

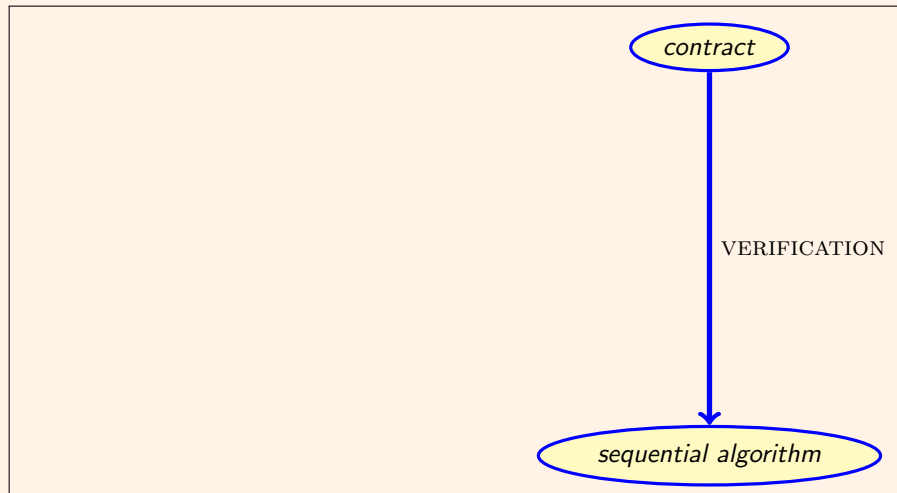
contract

The sequential pattern

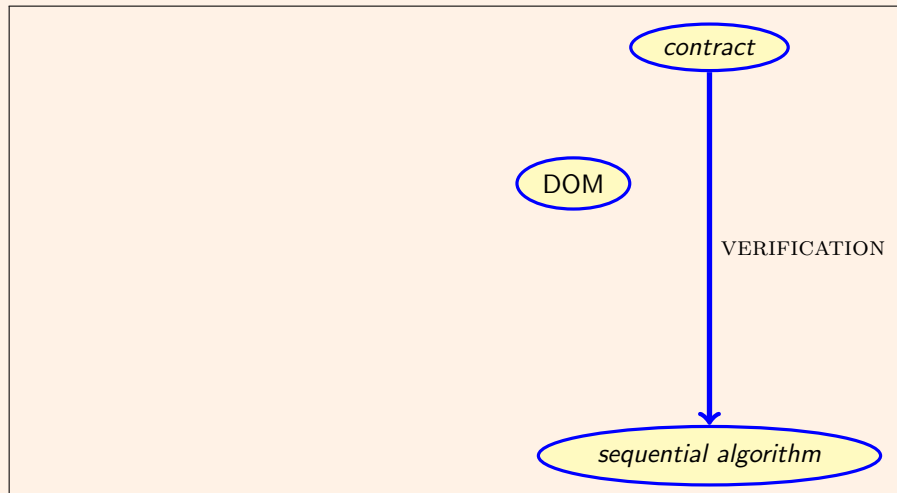
contract

sequential algorithm

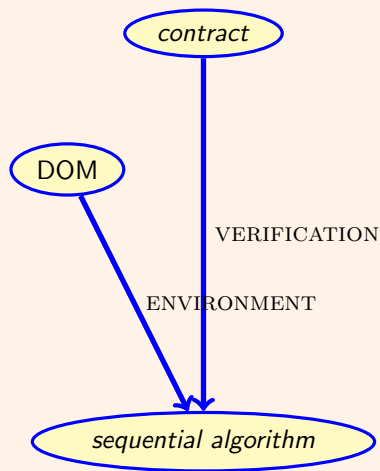
The sequential pattern



The sequential pattern

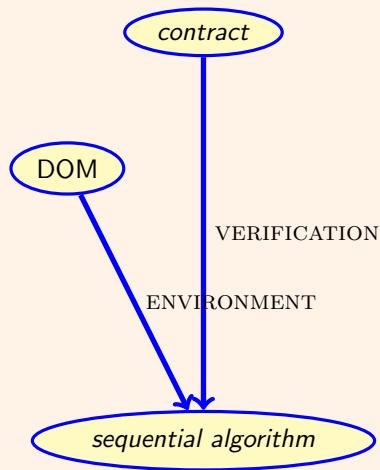


The sequential pattern

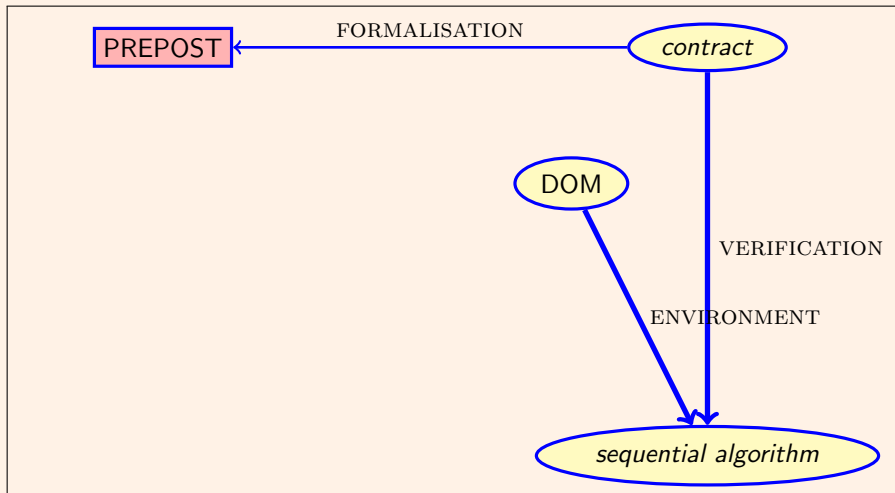


The sequential pattern

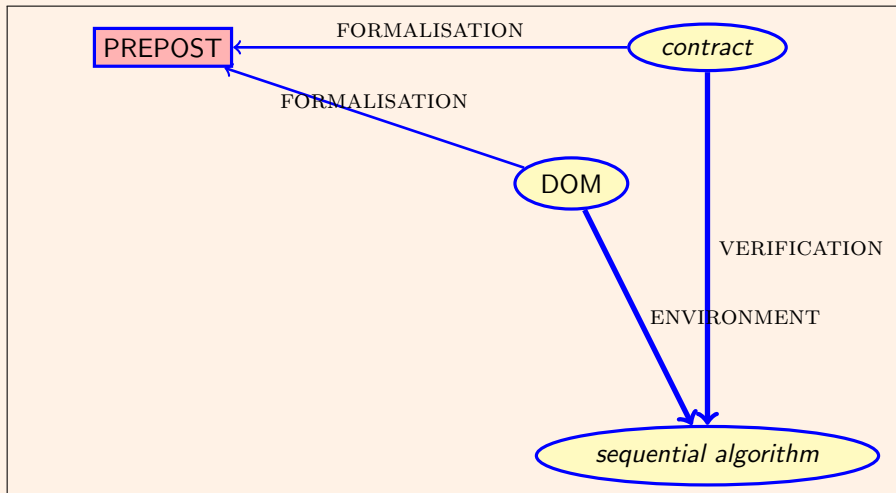
PREPOST



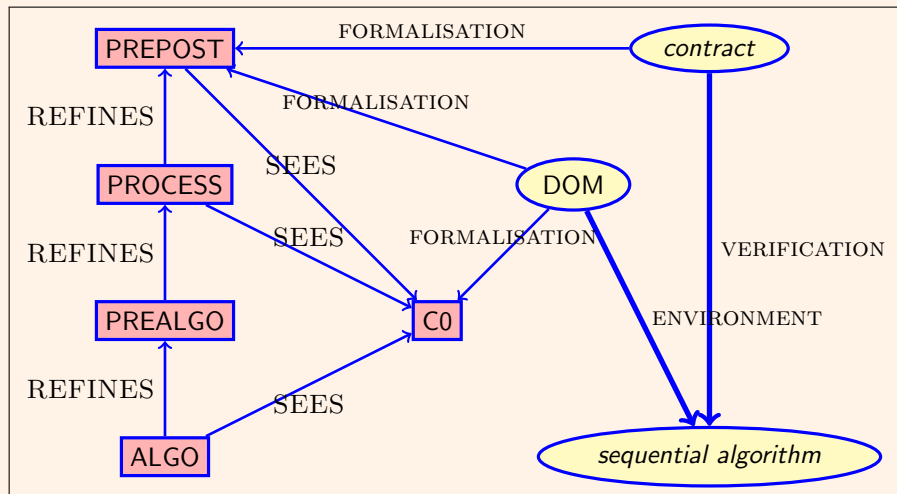
The sequential pattern



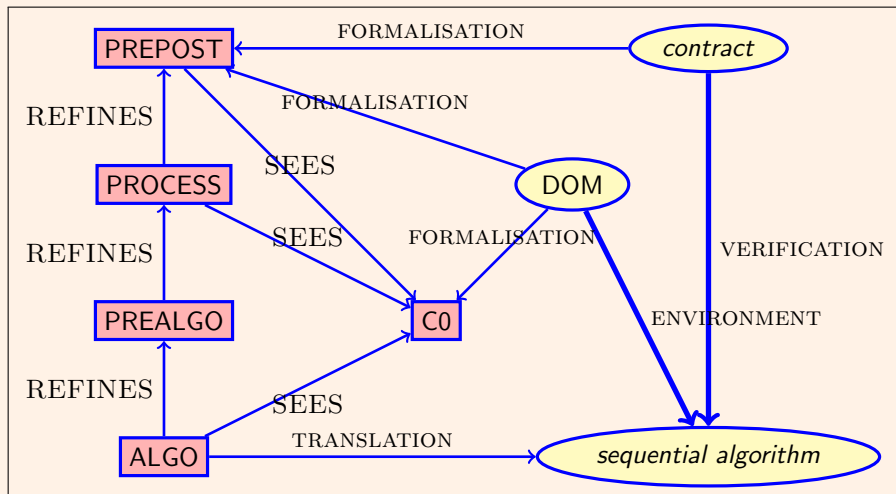
The sequential pattern



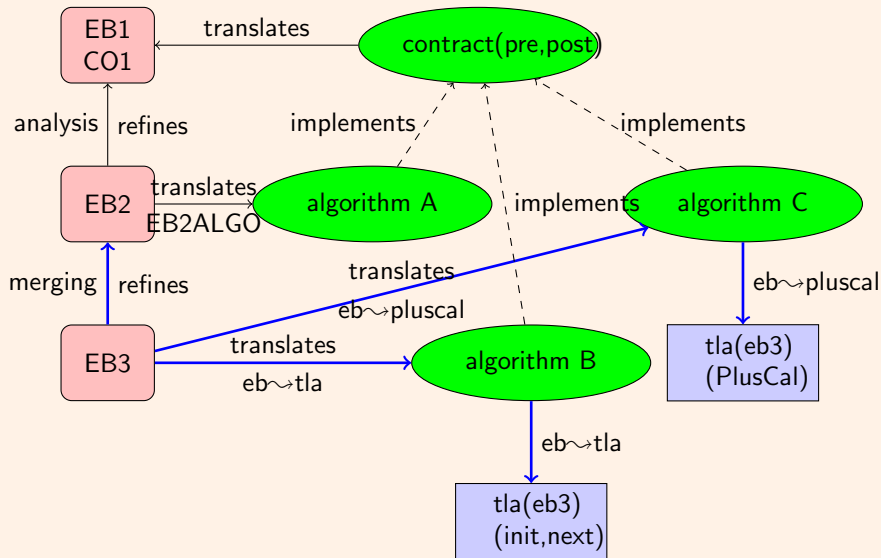
The sequential pattern



The sequential pattern



Global Picture



Current Section Summary

- 1 CM5
- 2 Overview
- 3 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
 - Analysis using Refinement
 - Synthesis by Merging
- 8 Summing a list of integer values

Short Summary on Event-B (context)

- Context : static properties of Event-B models
 - ▶ Sets : user-defined types
 - ▶ Constants : static object in development
 - ▶ Axioms : presumed properties about sets and constants
 - ▶ Theorems : derived properties about sets and constants

SETS

A

CONSTANTS

B, C, f

AXIOMS

$ax1 : B \subseteq A$

$ax2 : C \subseteq A$

$ax3 : g \in B \rightarrow C$

$ax4 : \forall A. A \subseteq \mathbb{N} \wedge 0 \in A \wedge suc[A] \subseteq A \Rightarrow \mathbb{N} \subseteq A$

...

Short Summary on Event-B (discrete)

- Machine : behavioral properties of Event-B models
 - Variables : states
 - Invariants : properties of variables that always need to hold
 - Theorems : derived properties about variables
 - Events : possible state changes
 - c et s are constantes and visible sets by e
 - x is a state variable or a list of variables
 - $G(c, s, t, x)$ is the condition for observing e .
 - $P(c, s, t, x, x')$ is the assertion for the relation over x and x' .
 - $BA(e)(c, s, x, x')$ is the *before-after* relationship for e and is defined by $\exists t. G(c, s, t, x) \wedge P(c, s, t, x, x')$.
- ```

EVENT e
ANY t
WHERE
 G(c, s, t, x)
THEN
 x : |(P(c, s, t, x, x'))
END

```

**EVENT**  $e$   
**ANY**  $t$   
**WHERE**  
 $G(c, s, t, x)$   
**THEN**  
 $x : |(P(c, s, t, x, x'))|$   
**END**

# Short Summary on Event-B (refinement)

- Given an **abstract** and a corresponding **concrete** event

**EVENT**  $ae \hat{=}$   
**ANY**  $v$  **WHERE**  
 $G(x, v)$   
**THEN**  
 $x := E(x, v)$   
**END**

**EVENT**  $ce \hat{=}$   
**ANY**  $w$  **WHERE**  
 $H(y, w)$   
**THEN**  
 $y := F(y, w)$   
**END**

$$\begin{aligned} I(x) \wedge J(x, y) \wedge H(y, w) \\ \implies \\ \exists v \cdot (G(x, v) \wedge J(E(x, v), F(y, w))) \end{aligned}$$

- $BA(ae)(x, x') \hat{=} \exists v. G(x, v) \wedge x' = E(x)$
- $BA(ce)(y, y') \hat{=} \exists w. H(y, w) \wedge y' = F(y)$

# Modelling systems in Event-B

# Modelling systems in Event-B

**MACHINE**

*m*

**SEES**

*c*

**VARIABLES**

*x*

**INVARIANT**

*I(x)*

**THEOREMS**

*Q(x)*

**INITIALISATION**

*Init(x)*

**EVENTS**

*...e*

**END**

# Modelling systems in Event-B

$c$  defines the static environment for the proofs  
related to  $m$  : sets, constants, axioms, theorems  $\Gamma(m)$ .

**MACHINE**

$m$

**SEES**

$c$

**VARIABLES**

$x$

**INVARIANT**

$I(x)$

**THEOREMS**

$Q(x)$

**INITIALISATION**

$Init(x)$

**EVENTS**

$\dots e$

**END**

# Modelling systems in Event-B

**MACHINE**

*m*

**SEES**

*c*

**VARIABLES**

*x*

**INVARIANT**

*I(x)*

**THEOREMS**

*Q(x)*

**INITIALISATION**

*Init(x)*

**EVENTS**

*...e*

**END**

*c* defines the static environment for the proofs related to *m* : sets, constants, axioms, theorems  $\Gamma(m)$ .

$$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$$

# Modelling systems in Event-B

**MACHINE**

*m*

**SEES**

*c*

**VARIABLES**

*x*

**INVARIANT**

*I(x)*

**THEOREMS**

*Q(x)*

**INITIALISATION**

*Init(x)*

**EVENTS**

*...e*

**END**

*c* defines the static environment for the proofs  
related to *m* : sets, constants, axioms, theorems  $\Gamma(m)$ .

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

$\forall e :$

$\Gamma(m) \vdash \forall x, x', u \in Values : I(x) \wedge R(u, x, x') \Rightarrow I(x')$

# Modelling systems in Event-B

**MACHINE**

*m*

**SEES**

*c*

**VARIABLES**

*x*

**INVARIANT**

*I(x)*

**THEOREMS**

*Q(x)*

**INITIALISATION**

*Init(x)*

**EVENTS**

*... e*

**END**

*c* defines the static environment for the proofs related to *m* : sets, constants, axioms, theorems  $\Gamma(m)$ .

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

$\forall e :$

$\Gamma(m) \vdash \forall x, x', u \in Values : I(x) \wedge R(u, x, x') \Rightarrow I(x')$

$\Gamma(m) \vdash \forall x \in Values : I(x) \Rightarrow Q(x)$

# Modelling systems in Event-B

**MACHINE**

*m*

**SEES**

*c*

**VARIABLES**

*x*

**INVARIANT**

*I(x)*

**THEOREMS**

*Q(x)*

**INITIALISATION**

*Init(x)*

**EVENTS**

*... e*

**END**

*c* defines the static environment for the proofs related to *m* : sets, constants, axioms, theorems  $\Gamma(m)$ .

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

$\forall e :$

$\Gamma(m) \vdash \forall x, x', u \in Values : I(x) \wedge R(u, x, x') \Rightarrow I(x')$

$\Gamma(m) \vdash \forall x \in Values : I(x) \Rightarrow Q(x)$

*e*

**ANY**

*u*

**WHERE**

*G(x, u)*

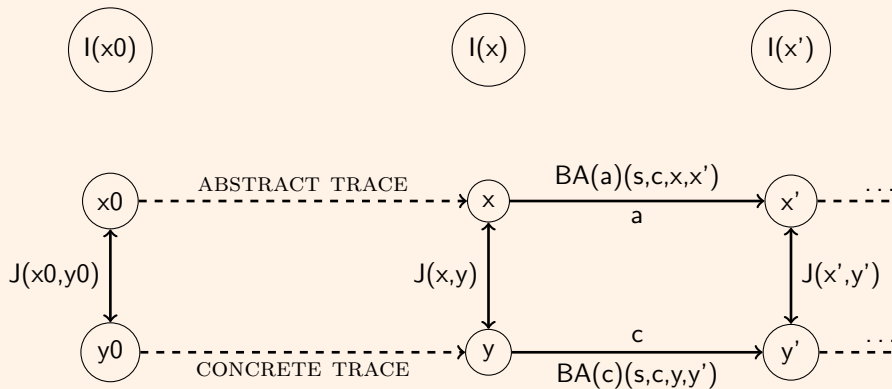
**THEN**

*x : |(R(u, x, x'))*

**END**

or *e* is **observed**  $x \xrightarrow{e} x'$

# Refinement between two machines



## Current Section Summary

- 1 CM5
- 2 Overview
- 3 Programming by contract
  - Short view
  - The call-as-event idea
  - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
  - Analysis using Refinement
  - Synthesis by Merging
- 8 Summing a list of integer values

# Merging two events

**EVENT** ae1

**ANY**  $u$  **WHERE**

$G1(u, s, c, x)$

**THEN**

$x : |ABAP(u, s, c, x, x')$

**END**

**EVENT** ae2

**ANY**  $u$  **WHERE**

$G2(u, s, c, x)$

**THEN**

$x : |ABAP(u, s, c, x, x')$

**END**

refines

refines

**EVENT** c

**REFINES** ae1, ae2

**ANY**  $u$  **WHERE**

$H(u, s, c, u)$

**THEN**

$x : |ABAP(u, s, c, x, x')$

**END**

# Merging two events

**EVENT** ae1

**ANY**  $u$  **WHERE**

$G1(u, s, c, x)$

**THEN**

$x : |ABAP(u, s, c, x, x')$

**END**

**EVENT** ae2

**ANY**  $u$  **WHERE**

$G2(u, s, c, x)$

**THEN**

$x : |ABAP(u, s, c, x, x')$

**END**

refines

refines

**EVENT** c

**REFINES** ae1, ae2

**ANY**  $u$  **WHERE**

$H(u, s, c, u)$

**THEN**

$x : |ABAP(u, s, c, x, x')$

**END**

$AX(s, c), I(s, c, x), H(v, s, c, y) \vdash G_1(s, c, x) \vee G_2(s, c, x).$

# Applying the fusion of events

## Strategy for merging events

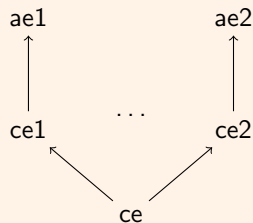
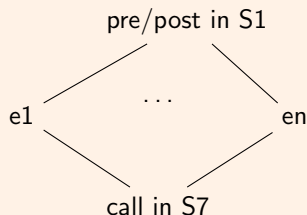
Let be two events  $ae1$  and  $ae2$  such that the two guards are complementary (ie  $g(ae1) \Leftrightarrow \neg g(ae2)$  or  $g(ae2) \Leftrightarrow \neg g(ae1)$  ).

- Let  $cei$  be the event with the guard  $g(aei)$  and with the action  $(g(ae1) \Rightarrow a(ae1)) \wedge (g(ae2) \Rightarrow a(ae2))$ .

**Then  $cei$  refines  $aei$ .**

- Let  $ce$  be the event with the guard  $g(ae1) \vee g(ae2)$ .

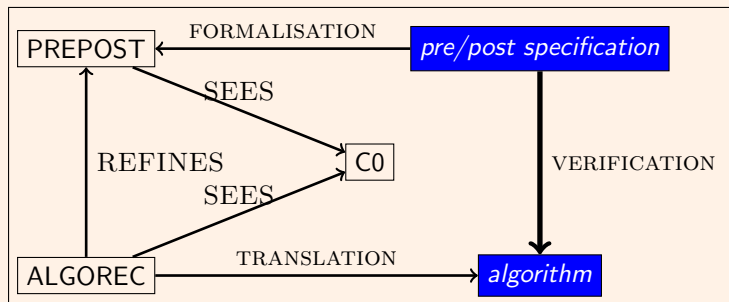
**Then  $ce$  refines  $cei$  by fusion.**



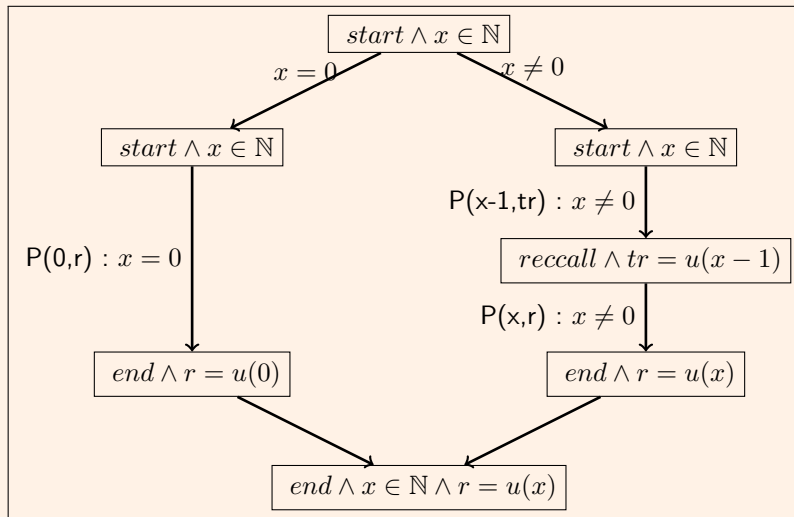
## Current Section Summary

- 1 CM5
- 2 Overview
- 3 Programming by contract
  - Short view
  - The call-as-event idea
  - Main idea
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
  - Analysis using Refinement
  - Synthesis by Merging
- 8 Summing a list of integer values

# The recursive pattern



# The refinement diagram



# Ideas for producing algorithms

- **EVENT**  $\text{rec}\% \text{PROC}(h(x), y)\%P(y)$  is simply simulating the recursive call of the same function.
- The invariant is defined in a simpler way by analysing the inductive structure and a control variable is introduced for structuring the inductive computation.

```
EVENT
e
WHERE
 $\ell = \ell_1$
 $g_{\ell_1, \ell_2}(x)$
THEN
 $\ell := \ell_2$
 $x := f_{\ell_1, \ell_2}(x)$
END
```

```
EVENT
 $\text{rec}\% \text{PROC}(h(x), y)\%P(y)$
ANY y
WHERE
 $\ell = \ell_1$
 $g_{\ell_1, \ell_2}(x, y)$
THEN
 $\ell := \ell_2$
 $x := f_{\ell_1, \ell_2}(x, y)$
END
```

```
EVENT
 $\text{call}\% \text{APROC}(h(x), y)\%P(y)$
ANY y
WHERE
 $\ell = \ell_1$
 $g_{\ell_1, \ell_2}(x, y)$
THEN
 $\ell := \ell_2$
 $x := f_{\ell_1, \ell_2}(x, y)$
END
```

# The computation of the function $x^2$

## VARIABLES

$r, c, tr$

## INVARIANTS

$inv1 : r \in \mathbb{N}$

$inv2 : c = end \Rightarrow r = n * n$

$inv3 : c = callrec \Rightarrow n \neq 0$

$inv4 : c = callrec \Rightarrow tr = (n - 1) * (n - 1)$

$inv5 : c \in C$

$inv6 : tr \in \mathbb{N}$

$inv7 : c = end \Rightarrow r = n * n$

$inv8 : c = end \wedge n \neq 0$

$\Rightarrow tr = (n - 1) * (n - 1) \wedge r = tr + 2 * (n - 1) + 1$

$inv9 : c = callrec \Rightarrow n * n = tr + 2 * (n - 1) + 1$

# The computation of the function $x^2$

```
EVENT INITIALISATION
BEGIN
 act1 : $r := 0$
 act2 : $c := start$
 act3 : $tr \in \mathbb{N}$
END
EVENT square0
REFINES square(n ; r)
WHEN
 grd1 : $c = start$
 grd2 : $n = 0$
THEN
 act1 : $c := end$
 act2 : $r := 0$
END
```

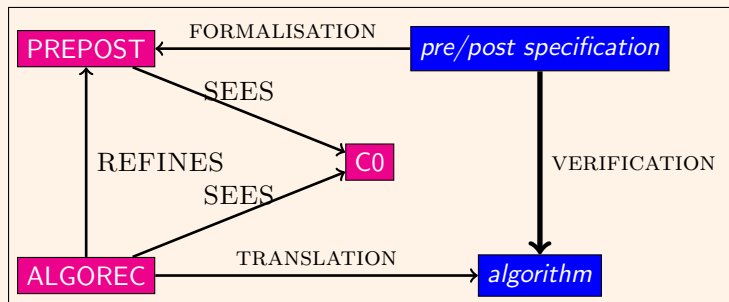
```
EVENT squaren
REFINES square(n ; r)
WHEN
 grd1 : $c = callrec$
THEN
 act1 : $r := tr + 2 * (n - 1) + 1$
 act2 : $c := end$
END
EVENT rec%square($n-1$; tr)
WHEN
 grd1 : $c = start$
 grd2 : $n \neq 0$
THEN
 act1 : $c := callrec$
 act2 : $tr := (n - 1) * (n - 1)$
END
```

## Summary for proof obligations

## Summary for proof obligations

| Name       | Total | Automatic | Interactive |
|------------|-------|-----------|-------------|
| cae-square | 34    | 32        | 2           |
| square0    | 3     | 2         | 1           |
| specsquare | 2     | 2         | 0           |
| square     | 29    | 28        | 1           |

# The recursive pattern



- Proofs are easier and simpler
- Invariant is simple to find
- Translation is automatic
- Students are not happy with it and tools are not always set for this verification.

## Current Section Summary

- ① CM5
- ② Overview
- ③ Programming by contract
  - Short view
  - The call-as-event idea
  - Main idea
- ④ Short Summary on Event-B
- ⑤ Internalizing events by merging
- ⑥ The Call-as-Event Paradigm
- ⑦ Analysis and then Synthesis
  - Analysis using Refinement
  - Synthesis by Merging
- ⑧ Summing a list of integer values

## Current Sub Section Summary

- 1 CM5
- 2 Overview
- 3 Programming by contract
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis  
Analysis using Refinement  
Synthesis by Merging
- 8 Summing a list of integer values

## First n numbers and first odd/even numbers

- First the **pre/post specification** ...
- Possibility to use a programming language with contracts too

- ### h contracts too

the sum of first  $n$   
 numbers and first even  
 smaller than  $n$

- sequences  $s$  and  $es$   
 the sum of first  $n$   
 numbers and first even  
 smaller than  $n$   
 :  
 $\mathbb{N} \Rightarrow s(n) = \sum_{i=0}^{i=n} i.$   
 $\mathbb{N} \Rightarrow es(n) =$   
 $2 * i.$



## First n numbers and first odd/even numbers

$$axm1 : n \in \mathbb{N}$$

$$axm2 : s \in \mathbb{N} \rightarrow \mathbb{N} \wedge os \in \mathbb{N} \rightarrow \mathbb{N} \wedge es \in \mathbb{N} \rightarrow \mathbb{N}$$

$$axm3 : es(0) = 0 \wedge os(0) = 0 \wedge s(0) = 0$$

$$axm4 : \forall i, l. i \in \mathbb{N} \wedge l \in \mathbb{N} \wedge i = 2 * l \Rightarrow s(i+1) = s(i) + i + 1 \wedge es(i+1) =$$

$$axm5 : \forall i, l. i \in \mathbb{N} \wedge l \in \mathbb{N} \wedge i = 2 * l + 1 \Rightarrow s(i+1) = s(i) + i + 1 \wedge es(i+1) =$$

$$axm6 : suc \in \mathbb{N} \rightarrow \mathbb{N} \wedge (\forall i. i \in \mathbb{N} \Rightarrow suc(i) = i + 1)$$

$$axm7 : \forall A. A \subseteq \mathbb{N} \wedge 0 \in A \wedge suc[A] \subseteq A \Rightarrow \mathbb{N} \subseteq A$$

$$th1 : \forall i. i \in \mathbb{N} \Rightarrow s(i+1) = s(i) + i + 1$$

$$th2 : \forall u, v. u \in \mathbb{N} \wedge v \in \mathbb{N} \wedge 2 * u = v \Rightarrow u = v/2$$

$$th3 : \forall k. k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k$$

$$th4 : \forall k. k \in \mathbb{N} \Rightarrow s(k) = (k * k + k)/2$$

$$th5 : \forall k. k \in \mathbb{N} \Rightarrow es(2 * k) = 2 * s(k)$$

$$th6 : \forall k. k \in \mathbb{N} \Rightarrow es(2 * k + 1) = 2 * s(k)$$

$$th7 : \forall k. k \in \mathbb{N} \wedge k \neq 0 \Rightarrow os(2 * k) = k * k$$

$$th8 : \forall k. k \in \mathbb{N} \Rightarrow os(2 * k + 1) = (k + 1) * (k + 1)$$

## INVARIANTS

$$inv1 : input \in \mathbb{Z}$$
$$inv6 : input = n$$
$$inv2 : rs \in \mathbb{Z} \wedge re \in \mathbb{Z}$$
$$inv3 : ok \in \mathit{BOOL}$$
$$inv4 : ok = TRUE \Rightarrow rs = s(input) \wedge re = es(input)$$

END

## Stating the pre/post specification in Event-B

## Refining for computing (1)

## INVARIANTS

$$inv1 : cur \in 0 .. n$$
$$inv2 : ee \in 0..n \mapsto \mathbb{N}$$
$$inv3 : ss \in 0..n \mapsto \mathbb{N}$$
$$inv5: dom(ss) = 0..cur \wedge dom(ee) = dom(ss) \wedge dom(ss) \subseteq \mathbb{N}$$
$$inv6 : \forall i. i \in 0 .. cur \Rightarrow ee(i) = es(i) \wedge ss(i) = s(i)$$

## VARIANT

## THEN

$$act1 : ok := FALSE$$
$$act2 : rs : \in \mathbb{Z}$$
$$act3 : re : \in \mathbb{Z}$$
$$act4 : input := n$$
$$act5 : ee := \{0 \mapsto 0\}$$
$$act6 : ss := \{0 \mapsto 0\}$$
$$act7 : cur := 0$$

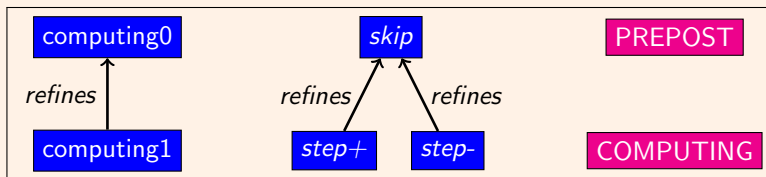
END

## Refining for computing (2)

## Refining for computing (3)

## Refining for computing (4)

# Diagram of events



## Completing the analysis

**MACHINE  $M_1$  SEES  $C_0$**

## VARIABLES

 $input, rs, re, ok,$ 

## INVARIANTS

$$inv1 : input \in \mathbb{Z}$$
$$inv6 : input = n$$
$$inv2 : rs \in \mathbb{Z} \wedge re \in \mathbb{Z}$$
$$inv3 : ok \in \mathit{BOOL}$$
$$inv4 : ok = TRUE \Rightarrow rs = s(input) \wedge re = es(input)$$

## Completing the analysis

MACHINE  $M_{11}$  SEES  $C_0$

## VARIABLES

 $input, rs, re, ok, ee, ss, cur,$ 

## INVARIANTS

$$inv1 : cur \in 0..n$$
$$inv2 : ee \in 0 .. n \mapsto \mathbb{N}$$
$$inv3 : ss \in 0 .. n \mapsto \mathbb{N}$$
$$inv5 : dom(ss) = 0 .. cur \wedge dom(ee) = dom(ss) \wedge dom(ss) \subseteq \mathbb{N}$$
$$inv6 : \forall i. i \in 0 .. cur \Rightarrow ee(i) = es(i) \wedge ss(i) = s(i)$$

## Completing the analysis

**MACHINE**  $M_{12}$  **SEES**  $C_0$

## VARIABLES

 $input, rs, re, ok, ee, ss, cur, cs, ce,$ 

## INVARIANTS

$$inv1 : cs = ss(cur)$$
$$inv2 : ce = ee(cur)$$

## Completing the analysis

**MACHINE**  $M_{13}$  **SEES**  $C_0$

## VARIABLES

 $input, rs, re, ok, ee, ss, cur, cs, ce,$ 

## INVARIANTS

$$inv1 : cs = ss(cur)$$
$$inv2 : ce = ee(cur)$$
$$inv4 : ce = es(cur)$$
$$inv3 : cs = s(cur)$$

## Completing the analysis

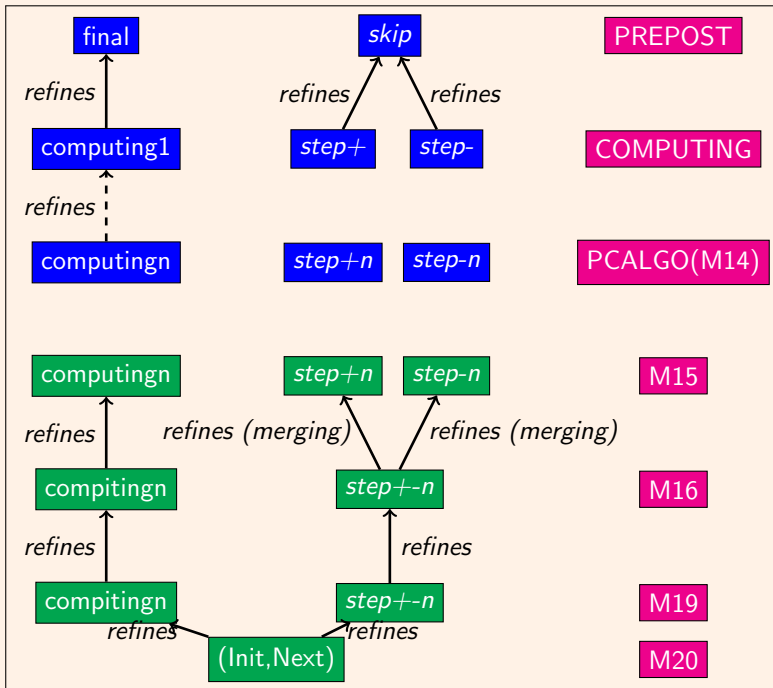
**MACHINE**  $M_{14}$  **SEES**  $C_0, C_1$

## VARIABLES

 $input, rs, re, cur, cs, ce, l,$ 

## INVARIANTS

$$inv1 : cs = s(cur)$$
$$inv5 : l = start \Leftrightarrow ok = FALSE$$
$$inv6 : l = end \Leftrightarrow ok = TRUE$$
$$inv2 : cs = s(cur)$$
$$inv3 : l \in L$$
$$inv4 : l = end \Rightarrow re = es(n) \wedge rs = s(n)$$



## Translation from Evenrts

```
struct sums codesum(int n)
{int k,ce,cs; struct sums r;
 r.s=0;r.se=0;k=0;ce=0;cs=0;
 while (k<n)
 {
 if (k % 2 != 0)
 { ce = ce + k + 1;cs = cs +k+1; k = k +1;}
 else
 { ce = ce ;cs = cs +k+1; k = k +1;}
 }
 r.s=cs;r.se=ce;
 return(r);
}
```

# Generation of an algorithm from M14

## Improving and checking with Frama-c

```
#include "structure.h"
#include "trans-even.h"
struct sums codesum(int n)
{
 int k, ce, cs; struct sums r;
 r.s=0; r.se=0; k=0; ce=0; cs=0;
 /*@ loop invariant k >= 0 && k <= n && mathsum(k) == cs;
 @ loop invariant ((k % 2 == 0) ==> (mathse(k) == ce));
 @ loop invariant ((k % 2 != 0) ==> (mathse(k) == ce));
 loop assigns k, ce, cs;
 loop variant n-k;
 */
 while (k < n)
 {
 if (k % 2 != 0)
 { ce = ce + k + 1; cs = cs + k + 1; k = k + 1; }
 else
 { ce = ce ; cs = cs + k + 1; k = k + 1; }
 }
 r.s=cs; r.se=ce;
 return(r);
}
```

# Generation of an algorithm from M14

## Improving and checking with Frama-c

```
#include "structure.h"
#include "trans-even.h"
struct sums codesum(int n)
{
 int k, ce, cs; struct sums r;
 r.s=0; r.se=0; k=0; ce=0; cs=0;
 /*@ loop invariant k >= 0 && k <= n && mathsum(k) == cs;
 @ loop invariant ((k % 2 == 0) ==> (mathse(k) == ce));
 @ loop invariant ((k % 2 != 0) ==> (mathse(k) == ce));
 loop assigns k, ce, cs;
 loop variant n-k;
 */
 while (k<n)
 {
 if (k % 2 != 0)
 { ce = ce + k + 1; }
 cs = cs +k+1; k = k +1;
 cp trabs-e* }
 r.s=cs; r.se=ce;
 return(r);
}
```

# Generation of an algorithm from M14

Using the Plugin EB2Algo

```
rs : $\in \mathbb{Z}$ ||
re : $\in \mathbb{Z}$ ||
input = n ||
cur = 0 ||
ce = 0 ||
cs = 0

while cur $\neq$ n do
 if cur mod 2 $\neq$ 0 then
 cur = cur+1 ||
 cs = cs+cur+1 ||
 ce = ce+cur+1
 else
 cur = cur+1 ||
 cs = cs+cur+1 ||
 ce = ce
```

## Current Sub Section Summary

- 1 CM5
- 2 Overview
- 3 Programming by contract
- 4 Short Summary on Event-B
- 5 Internalizing events by merging
- 6 The Call-as-Event Paradigm
- 7 Analysis and then Synthesis
  - Analysis using Refinement
  - Synthesis by Merging
- 8 Summing a list of integer values

## Merging events

```

EVENT e1
ANY
 x
WHERE
 $G1(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))|$
END

```

```

EVENT e2
ANY
 x
WHERE
 $G2(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))|$
END

```

# Merging events

```
EVENT e1
ANY
 x
WHERE
 $G1(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))$
END
```

```
EVENT e2
ANY
 x
WHERE
 $G2(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))$
END
```

```
EVENT $e \text{ refines } e1, e2$
ANY
 x
WHERE
 $H(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))$
END
```

# Merging events

```
EVENT e1
ANY
 x
WHERE
 $G1(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))$
END
```

```
EVENT e2
ANY
 x
WHERE
 $G2(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))$
END
```

```
EVENT $e \text{ refines } e1, e2$
ANY
 x
WHERE
 $H(s, c, x, u)$
THEN
 $u : |(R(s, c, x, u, u'))$
END
```

Checking the Proof Obligation :

$$Ax(s, c), I(s, c, u), H(s, c, x, u) \vdash \\ G1(s, c, x, u) \vee G2(s, c, x, u)$$

- Preparing merging by transforming actions.

**EVENT** e

**ANY**

$x$

**WHERE**

$G(s, c, x, u)$

**THEN**

$u : |(R(s, c, x, u, u'))$

**END**

**EVENT** f refines e

**ANY**

$x$

**WHERE**

$G(s, c, x, u)$

**THEN**

$u : |(G(s, c, x, u) \Rightarrow R(s, c, x, u, u'))$

**END**

- Internalizing the action as a before after relation
- $G(s, c, x, u) \wedge (G(s, c, x, u) \Rightarrow R(s, c, x, u, u')) \Leftrightarrow$   
 $G(s, c, x, u) \wedge R(s, c, x, u, u')$

- Preparing merging by transforming actions.

**EVENT** e

**ANY**

*x*

**WHERE**

$G(s, c, x, u)$

**THEN**

$u : |(R(s, c, x, u, u'))$

**END**

**EVENT** f refines e

**ANY**

*x*

**WHERE**

$G(s, c, x, u)$

**THEN**

$u : |(G(s, c, x, u) \Rightarrow R(s, c, x, u, u'))$

**END**

- $G(s, c, x, u) \wedge (G(s, c, x, u) \Rightarrow R(s, c, x, u, u')) \Leftrightarrow G(s, c, x, u) \wedge R(s, c, x, u, u')$
- $A(s, c, u) \wedge I(s, c, u) \wedge BA(f)(s, c, u, u') \Rightarrow I(s, c, u') \wedge BA(e)(s, c, u, u')$

- Preparing merging by transforming actions.

**EVENT** e

**ANY**

*x*

**WHERE**

$G(s, c, x, u)$

**THEN**

$u : |(R(s, c, x, u, u'))$

**END**

**EVENT** f refines e

**ANY**

*x*

**WHERE**

$G(s, c, x, u)$

**THEN**

$u : |(G(s, c, x, u) \Rightarrow R(s, c, x, u, u'))$

**END**

- $G(s, c, x, u) \wedge (G(s, c, x, u) \Rightarrow R(s, c, x, u, u')) \Leftrightarrow$   
 $G(s, c, x, u) \wedge R(s, c, x, u, u')$
- $A(s, c, u) \wedge I(s, c, u) \wedge BA(f)(s, c, u, u') \Rightarrow$   
 $I(s, c, u') \wedge BA(e)(s, c, u, u')$
- $A(s, c, u) \wedge I(s, c, u) \wedge G(s, c, x, u) \wedge (G(s, c, x, u) \Rightarrow$   
 $R(s, c, x, u, u')) \Rightarrow I(s, c, u') \wedge G(s, c, x, u) \wedge R(s, c, x, u, u')$

## Synthesis Phase : review of machines from M15-M20

**MACHINE** *M15* **SEES** *C0, C1*  
**VARIABLES**  
*input, rs, re, cur, cs, ce, l,*

## Synthesis Phase : review of machines from M15-M20

**MACHINE** *M16* **SEES** *C0, C1*  
**VARIABLES**  
*input, rs, re, cur, cs, ce, l,*

# Synthesis Phase : review of machines from M15-M20

**MACHINE**  $M17$  **SEES**  $C0, C1$   
**VARIABLES**

$input, rs, re, cur, cs, ce, l,$

## Synthesis Phase : review of machines from M15-M20

**MACHINE** *M18* **SEES** *C0, C1*  
**VARIABLES**  
*input, rs, re, cur, cs, ce, l,*

**MACHINE** *M19* **SEES** *C0, C1*  
**VARIABLES**  
*input, rs, re, cur, cs, ce, l,*

## Synthesis Phase : review of machines from M15-M20

**MACHINE** *M20* **SEES** *C0, C1*  
**VARIABLES**  
*input, rs, re, cur, cs, ce, l,*

# Generating a TLA specification

```

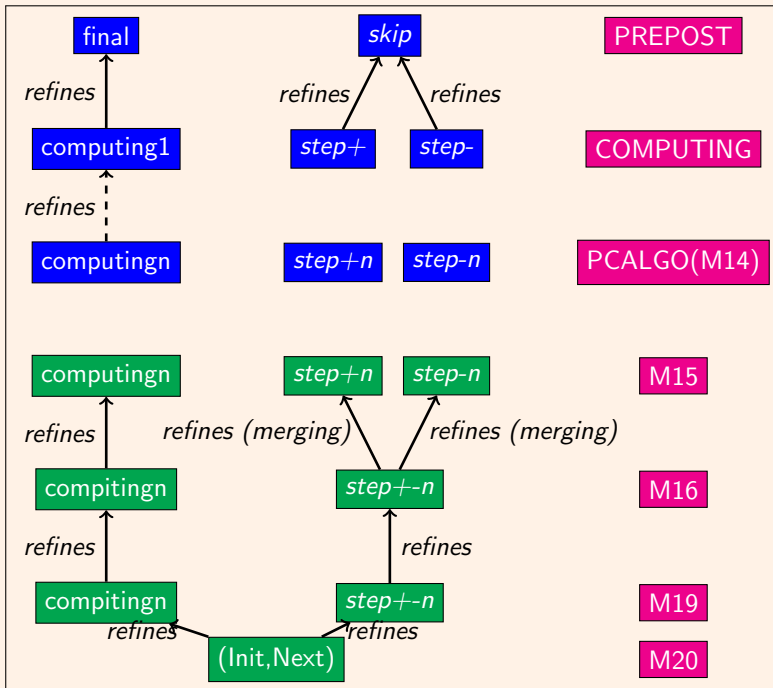
next1 ==
 (l="start" /\ cur < n /\ (cur % 2 = 0))
 /\ (cur'=cur+1 /\ ce'=ce /\ l'=l
 /\ cs'=cs+cur+1 /\ re'=re /\ rs'=rs)
o
delsnext2 ==
 (l="start" /\ cur < n /\ (cur % 2 # 0))
 /\ (cur'=cur+1 /\ ce'=ce+cur+1 /\ l'=l
 /\ cs'=cs+cur+1 /\ re'=re /\ rs'=rs)

next3 ==
 (l="start" /\ cur=n) /\ (rs'=cs /\ re'=ce
 /\ l'="end" /\ cur'=cur /\ cs'=cs /\ ce'=ce)

Next ==
 (next1 \/ next2 \/ next3)

Init == l="start" /\ cur=0 /\ rs=0 /\ cs=0 /\ re=0 /\ ce=0
=====

```



## Current Section Summary

- ① CM5
- ② Overview
- ③ Programming by contract
  - Short view
  - The call-as-event idea
  - Main idea
- ④ Short Summary on Event-B
- ⑤ Internalizing events by merging
- ⑥ The Call-as-Event Paradigm
- ⑦ Analysis and then Synthesis
  - Analysis using Refinement
  - Synthesis by Merging
- ⑧ Summing a list of integer values

# Contract for the problem Summing a list of integer values

Contract for the problem Summing  
a list of integer values

**VARIABLES**  $n, v, r, done$

---

**REQUIRES**  $\left\{ \begin{array}{l} n_0 \in \mathbb{N} \\ n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \\ r_0 \in \mathbb{Z} \\ done_0 \in \text{BOOL} \end{array} \right.$

**ENSURES**  $\left( \begin{array}{l} r_f = \sum_{k=1}^{n_0} v_0(k) \\ done_f = \text{TRUE} \end{array} \right)$

---

## Contract for the problem Summing a list of integer values

Contract for the problem Summing  
a list of integer values

**VARIABLES**  $n, v, r, done$

$$\begin{array}{l} \text{REQUIRES} \left\{ \begin{array}{l} n_0 \in \mathbb{N} \\ n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \\ r_0 \in \mathbb{Z} \\ done_0 \in \text{BOOL} \end{array} \right. \\ \text{ENSURES} \left( \begin{array}{l} r_f = \sum_{k=1}^{n_0} v_0(k) \\ done_f = \text{TRUE} \end{array} \right) \end{array}$$

- the initial values of  $n$ ,  $v$  and  $done$ , denoted  $n_0$ ,  $v_0$ ,  $done_0$  and the final value of  $r$  denoted  $r_f$ .
- the final value of the variable  $r$  denoted  $r_f$  is the sum  $\sum_{k=1}^{n_0} v_0(k)$ .

# Contract for the problem Summing a list of integer values

Contract for the problem Summing a list of integer values

**VARIABLES**  $n, v, r, done$

**REQUIRES**  $\left\{ \begin{array}{l} n_0 \in \mathbb{N} \\ n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \\ r_0 \in \mathbb{Z} \\ done_0 \in \text{BOOL} \end{array} \right.$

**ENSURES**  $\left( \begin{array}{l} r_f = \sum_{k=1}^{n_0} v_0(k) \\ done_f = \text{TRUE} \end{array} \right.$

- the initial values of  $n$ ,  $v$  and  $done$ , denoted  $n_0$ ,  $v_0$ ,  $done_0$  and the final value of  $r$  denoted  $r_f$ .
- the final value of the variable  $r$  denoted  $r_f$  is the sum  $\sum_{k=1}^{n_0} v_0(k)$ .

## computation method

Defining the computation process by defining a sequence  $u$  in the context  $S_0$

```
CONTEXT S0
CONSTANTS n0 v0 u
AXIOMS
 @pre1 n0 ∈ ℕ
 @pre2 n0 ≠ 0
 @pre5 v0 ∈ 1..n0 → ℤ
 @axm1 u ∈ 0..n0 → ℤ
 @axm2 u(0) = 0
 @axm3 ∀ k. k ∈ 1..n0
 ⇒ u(k) = u(k - 1) + v0(k)
end
```

```
CONTEXT S0
CONSTANTS n0 v0 u
AXIOMS
@pre1 n0 ∈ ℕ
@pre2 n0 ≠ 0
@pre5 v0 ∈ 1..n0 → ℤ
@axm1 u ∈ 0..n0 → ℤ
@axm2 u(0) = 0
@axm3 ∀ k. k ∈ 1..n0
 ⇒ u(k) = u(k - 1) + v0(k)
end
```

- The context S0 defines the sequence  $u$  supporting the computation method by definition of the summation expression.

```
CONTEXT S0
CONSTANTS n0 v0 u
AXIOMS
 @pre1 n0 ∈ ℕ
 @pre2 n0 ≠ 0
 @pre5 v0 ∈ 1..n0 → ℤ
 @axm1 u ∈ 0..n0 → ℤ
 @axm2 u(0) = 0
 @axm3 ∀ k. k ∈ 1..n0
 ⇒ u(k) = u(k - 1) + v0(k)
end
```

- The context S0 defines the sequence  $u$  supporting the computation method by definition of the summation expression.
- The refinement process starts by the specification namely S1 and leads to the machine S7 which is containing the event final internalizing the computation of the sum.

# Expressing pre/post in a machine S1 (I)

```
MACHINE S1 SEES S0
VARIABLES r v n done
INVARIANTS
 @inv1 r ∈ ℤ
 @inv2 n ∈ ℤ
 @inv3 v ∈ 1..n → ℤ
 @inv4 done ∈ BOOL
 @read-values n = n0 ∧ v = v0
 @post done = TRUE
 ⇒ r = u(n0)
EVENT INITIALISATION
 then
 @act1 r : ∈ ℤ
 @act2 n := n0
 @act3 v := v0
 @act4 done := FALSE
 end
```

- $v$  and  $n$  are variables getting the input values as  $v0$  and  $n0$  : *read-values* label.
- $r$  is getting the result of the computing process under the control of *done* : post label.
- *post* : expression of the partial correctness.

# Expressing pre/post in a machine S1 (II)

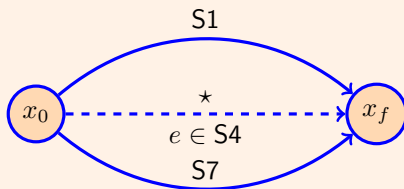
- The *final1* event expresses the relationship between initial and final values of variables.
- The *keep* event expresses the *hidden* loop preparing the final result.

```
EVENT final1
 where
 @grd1 done = FALSE
 then
 @act1 r := u(n)
 @act2 done := TRUE
 end

 anticipated EVENT keep
 then
 @act1 r, n, v, done :|
 (r' ∈ ℤ ∧ n' ∈ ℤ
 ∧ v' ∈ 1..n' → ℤ
 ∧ done' ∈ BOOL
 ∧ n' = n0 ∧ v' = v0
 ∧ (done' = TRUE
 ⇒ r' = u(n0)))
 end
```

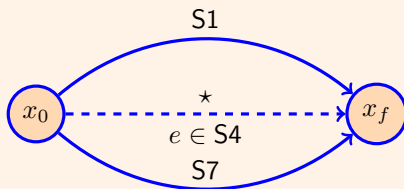
# Summary of the refinement steps

Refinement from S1  
till S7.



# Summary of the refinement steps

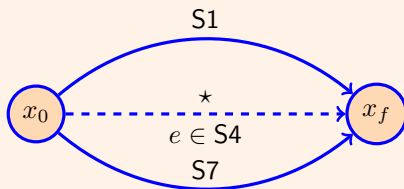
Refinement from S1  
till S7.



- Identifying the final1 event as simulating the pre/post contract in S1.

# Summary of the refinement steps

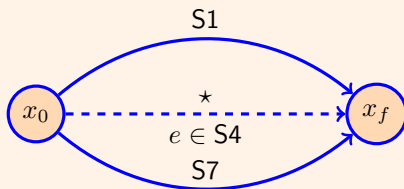
Refinement from S1  
till S7.



- Identifying the final event as simulating the pre/post contract in S1.
- Refining by using  $u$  and introducing events simulating computing steps in S4.

# Summary of the refinement steps

Refinement from S1  
till S7.



- Identifying the final event as simulating the pre/post contract in S1.
- Refining by using  $u$  and introducing events simulating computing steps in S4.
- Merging events of S4 into one event in S7.

## From S1 to S4 via S2 and S3

# From S1 to S4 via S2 and S3

```

MACHINE S2 REFINES S1 SEES S0
VARIABLES r i uu n v done
INVARIANTS
 @inv1 $i \in 0..n$
 @inv2 $uu \in 0..n \rightarrow \mathbb{Z} \wedge \text{dom}(uu) = 0..i$
 @inv3 $\forall k. k \in \text{dom}(uu) \Rightarrow uu(k) = u(k)$
 @inv4 $\text{done} = \text{FALSE} \Rightarrow i \in 0..n$
 @inv6 $\text{done} = \text{TRUE} \Rightarrow i = n$
 @inv7 $i \in 0..n - 1 \Rightarrow \text{done} = \text{FALSE}$
theorem @datath $v = v0 \wedge n = n0$
VARIANT $n - i$

```

- The variable  $uu$  is recording the sequence  $u$  by computing until  $i$  is  $n0$ .
- The new event step is refining the event  $keep$  and decreases the variant.

# From S1 to S4 via S2 and S3

```

MACHINE S2 REFINES S1 SEES S0
VARIABLES r i uu n v done
INVARIANTS
 @inv1 $i \in 0..n$
 @inv2 $uu \in 0..n \rightarrow \mathbb{Z} \wedge \text{dom}(uu) = 0..i$
 @inv3 $\forall k. k \in \text{dom}(uu) \Rightarrow uu(k) = u(k)$
 @inv4 $done = FALSE \Rightarrow i \in 0..n$
 @inv6 $done = TRUE \Rightarrow i = n$
 @inv7 $i \in 0..n - 1 \Rightarrow done = FALSE$
theorem @datath $v = v0 \wedge n = n0$
VARIANT $n - i$

```

```

MACHINE S3 REFINES S2 SEES S0
VARIABLES r i uu cu n v done
INVARIANTS
@inv1 cu ∈ ℤ
@inv2 cu = u(i)

```

- The variable  $uu$  is recording the sequence  $u$  by computing until  $i$  is  $n0$ .
- The new event step is refining the event `keep` and decreases the variant.
- The new variable  $cu$  is recording the last computed value of  $uu$ .
- Proofs of the invariants are made easier.

# The machine $S_4$ in the middle (I)

```
MACHINE S_4 REFINES S_3 SEES S_0
VARIABLES $r\ i\ cu\ n\ v\ done$
INVARIANTS
 theorem @inv1 $cu = u(i)$
EVENTS
 EVENT INITIALISATION
 then
 @act1 $r : \in \mathbb{Z}$
 @act2 $i := 0$
 @act4 $cu := 0$
 @act5 $v := v0$
 @act6 $n := n0$
 @act7 $done := FALSE$
 end
```

# The machine S4 in the middle (I)

```
MACHINE S4 REFINES S3 SEES S0
VARIABLES r i cu n v done
INVARIANTS
 theorem @inv1 cu = u(i)
EVENTS
 EVENT INITIALISATION
 then
 @act1 r :∈ ℤ
 @act2 i := 0
 @act4 cu := 0
 @act5 v := v0
 @act6 n := n0
 @act7 done := FALSE
 end
```

- The variable *uu* helps in the proof process but is expensive.
- The variable *uu* is hidden in the abstractions.

# The machine S4 in the middle (II)

```
EVENT final REFINES final
 where
 @grd1 i = n
 @grd2 done = FALSE
 then
 @act1 r := cu
 @act2 done := TRUE
 end

EVENT step000 REFINES step
 where
 @grd1 i < n
 @grd2 done = FALSE
 then
 @act1 i := i + 1
 @act3 cu := cu + v(i + 1)
 end
```

## The machine S4 in the middle (II)

```

EVENT final REFINES final
 where
 @grd1 $i = n$
 @grd2 $done = FALSE$
 then
 @act1 $r := cu$
 @act2 $done := TRUE$
 end
end

```

- Events are made simpler and  $uu$  is hidden.
- Events are guarded by evaluable conditions as  $i < n$  which is implying  $i \notin \text{dom}(uu)$
- Using EB2ALGO, one gets an algorithm but no proof is given of the transformations.

# The machine S7 with one event

```
MACHINE S7 REFINES S6 SEES S0
VARIABLES r i cu n v done
INVARIANTS
 theorem @inv1 done = FALSE $\Rightarrow$ $i \in 0..n$
 theorem @inv2 done = TRUE $\Rightarrow$ $i = n$
 theorem @inv3 $i \in 0..n - 1 \Rightarrow$ done = FALSE
 theorem @inv4 $i \in 0..n$
EVENT final REFINES final
 where
 @grd2 done = FALSE
 then
 @act1 r, i, cu, n, v, done :| (
 (i = n
 $\Rightarrow$ ($r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n$
 $\wedge v' = v \wedge done' = TRUE$))
 $\wedge (i < n$
 $\Rightarrow$ ($r' = r \wedge cu' = cu + v(i + 1) \wedge i' = i + 1$
 $\wedge n' = n \wedge v' = v \wedge done' = done$))
)
 end
```

## Defining an algorithm by a set of actions

## Defining an algorithm by a set of actions

$$guard(r, i, cu, n, v, done) \equiv done = FALSE$$

$$action \equiv \begin{cases} i = n \Rightarrow \begin{cases} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \\ r' = r \wedge cu' = cu + v(i+1) \wedge i' = i+1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{cases} \\ i < n \Rightarrow \end{cases}$$

## Defining an algorithm by a set of actions

$$guard(r, i, cu, n, v, done) \equiv done = FALSE$$

$$action \equiv \begin{cases} i = n \Rightarrow \begin{cases} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \end{cases} \\ i < n \Rightarrow \begin{cases} r' = r \wedge cu' = cu + v(i+1) \wedge i' = i+1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{cases} \end{cases}$$

- The machine  $S7$  contains an event Initialisation and a next relation
-

## Defining an algorithm by a set of actions

$$\begin{aligned} guard(r, i, cu, n, v, done) &\equiv done = FALSE \\ action &\equiv \begin{cases} i = n \Rightarrow \begin{cases} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \end{cases} \\ i < n \Rightarrow \begin{cases} r' = r \wedge cu' = cu + v(i+1) \wedge i' = i+1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{cases} \end{cases} \end{aligned}$$

- The machine  $S7$  contains an event Initialisation and a next relation
- 

## Definition

(mapping of a binary relation)

A binary relation  $R$  on  $State$  can be mapped to a GKAT term  $e$  for the interpretation  $i$ , if  $\mathcal{R}_i(e) = R$  and we write  $map_i(R) = e$

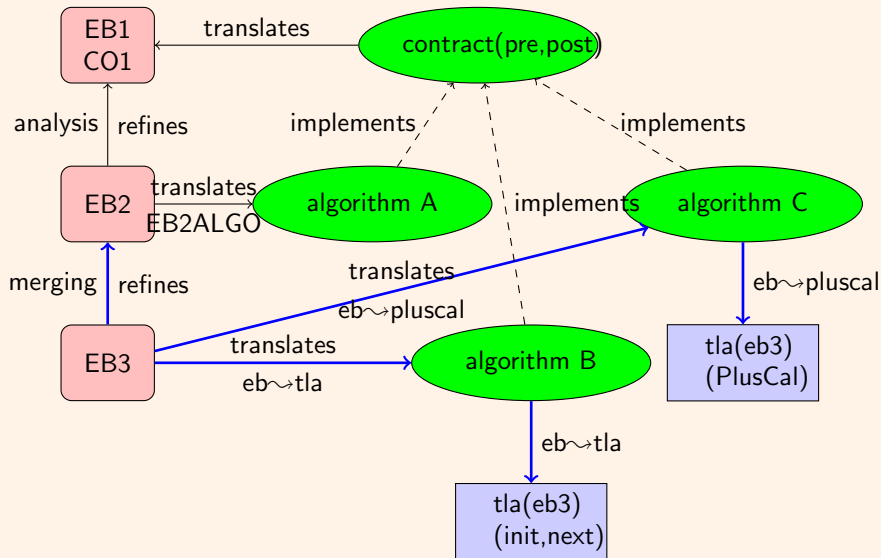


# Defining an algorithm by a set of actions (II)

Listing 4 – PlusCal algorithm for summing a vector

```
—algorithm sumvect {
variables n=n0, v=v0, i=0, cu=0, r;
{
w1: while (i # n) {
 w2: cu:=cu+v[i+1]; i:=i+1;
};
w3: r:=cu; Gu
}}
```

# Global View



- Algorithm A is generated using EB2Algo

- $$\begin{array}{l} r := \in \mathbb{Z} \parallel i := 0 \parallel cu := 0 \parallel v := v0 \parallel n := n0 \\ \text{while } i < n \text{ do} \\ \quad i := i + 1 \parallel cu := cu + v(i + 1) \\ \text{od;} \end{array}$$

- Algorithm B is simply defined by

$$guard(r, i, cu, n, v, done) \equiv done = FALSE$$

$$action \equiv \begin{cases} i = n \Rightarrow \begin{cases} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \\ r' = r \wedge cu' = cu + v(i+1) \wedge i' = i+1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{cases} \\ i < n \Rightarrow \end{cases}$$

Algorithm C is generated using our transformations based on GKAT algebra

```

--algorithm sumvect {
variables n=n0,v=v0,i=0,cu=0,r;
{
w1:while (i # n) {
 w2:cu:=cu+v[i+1]; i:=i+1;
};
w3:r:=cu;
}}

```